

10장 프로그래밍 언어 활용



340383

23.7, 20.10



211 헝가리안 표기법



헝가리안 표기법(Hungarian Notation)이란 변수명 작성 시 변수의 자료형을 알 수 있도록 자료형을 의미하는 문자를 포함하여 작성하는 방법이다. 예를 들어 정수형 변수라는 것을 알 수 있도록 변수명에 int를 의미하는 i를 덧붙여 i_InputA, i_InputB, i_Result라고 하는 것과 같다.

예 int i_InputA : 정수형 변수

예 double d_Result : 배정도 실수형 변수

340384

필기 25.5, 25.2, 21.3, 20.8



212 주요 자료형



종류	자료형	크기(C)	크기(Java)
필기 20.8 정수형	int	4Byte	4Byte
필기 25.5, 21.3 문자형	char	1Byte	2Byte
실수형	float	4Byte	4Byte
	double	8Byte	8Byte

340387

21.10, 필기 25.2, 24.2



213 연산자 우선순위



대분류	중분류	연산자	결합규칙	우선 순위
단항 연산자	단항 연산자	!(논리 not) ~(비트 not) ++(증가) --(감소) sizeof(기타)	←	높음 ↑
이항 연산자	산술 연산자	* / %(나머지)	→	
		+ -		
	시프트 연산자	<< >>		
	관계 연산자	< <= >= >		
		=(같다) !=(같지 않다)		
	비트 연산자	&(비트 and) ^(비트 xor) (비트 or)		
논리 연산자		&&(논리 and) (논리 or)		
삼항 연산자	조건 연산자	? :	→	낮음 ↓
대입 연산자	대입 연산자	= += -= *= /= %= << >>= 등	←	
순서 연산자	순서 연산자	,	→	



214 제어문 - C언어



예제 다음 C언어로 구현된 프로그램의 실행 결과를 확인하시오.

```
#include <stdio.h>

main() {
    ① int score[] = { 86, 53, 95, 76, 61 };
    ② char grade;
    ③ char str[] = "Rank";
    ④ ⑪ ⑮ for (int i = 0; i < 5; i++) {
    ⑤ ⑫ switch (score[i] / 10) {
        case 10: 'score[i]/10'이 10일 경우 찾아오는 곳으로, 'A'를 저장하기 위해 그냥 다음 문장으로 이동한다.
        case 9: 'score[i]/10'이 9일 경우 찾아오는 곳이다.
                grade = 'A'; grade에 'A'를 저장한다.
                break; switch문을 벗어나 ⑨ ⑭번으로 이동한다.

    ⑥ case 8:
    ⑦ grade = 'B';
    ⑧ break;

        case 7: 'score[i]/10'이 7일 경우 찾아오는 곳이다.
                grade = 'C'; grade에 'C'를 저장한다.
                break; switch문을 벗어나 ⑨ ⑭번으로 이동한다.

    ⑬ default: grade = 'F';
    }
    ⑨ ⑭ if (grade != 'F')
    ⑩ printf("%d is %c %s\n", i + 1, grade, str);
    }
}
```

코드 해설

① int score[] = { 86, 53, 95, 76, 61 };

5개의 요소를 갖는 정수형 배열 score를 선언한다. 개수를 지정하지 않았으므로, 초기값으로 지정된 수만큼 배열의 요소가 만들어진다.

	[0]	[1]	[2]	[3]	[4]
score	86	53	95	76	61

• int : 배열에 저장할 자료의 형이다.

- score : 사용할 배열의 이름으로, 사용자가 임의로 지정한다.
- [] : 배열의 크기를 지정하는 것으로, 개수를 지정하지 않으면, 초기값으로 지정된 수만큼 배열의 요소가 만들어진다.

② 문자형 변수 grade를 선언한다.

③ char str[] = "Rank";

- 문자형 배열 str을 선언하고 "Rank"로 초기화한다.
- 문자열을 저장하는 경우 문자열의 끝을 의미하는 널 문자('\0')가 추가로 저장되며, 출력 시 널 문자는 표시되지 않는다.

	[0]	[1]	[2]	[3]	[4]
str	R	a	n	k	\0

④ for (int i = 0; i < 5; i++)

반복 변수 i가 0에서 시작하여 1씩 증가하면서 5보다 작은 동안 for의 { } 부분을 반복 수행한다. 즉 for의 { } 부분인 ⑤~⑩번 문장을 5번 반복 수행한다.

- for는 반복문을 의미하는 예약어로 그대로 입력한다.
- i = 0 : 초기값을 지정할 수식이다.
- i < 5 : 최종값으로 지정할 수식이다.
- i++ : 증가값으로 사용할 수식이다.

⑤ switch (score[i] / 10)

score[i]를 10으로 나눈 결과에 해당하는 숫자를 찾아간다. 현재 i가 00이므로 score[0]인 86을 10으로 나눈 결과는 8.6이지만 C, Java에서 정수형 변수에 저장된 값의 나눗셈은 결과도 정수가 되므로 결과는 8이다. 8에 해당하는 ⑥번으로 이동한다.

- switch는 switch문에 사용되는 예약어로, 그대로 입력한다.
- score[i]/10 : case들의 값 중 하나를 도출하는 수식이다. 변수를 적어도 된다.

⑥ 'score[i]/10'이 8일 경우 찾아오는 곳이다.

⑦ grade에 'B'를 저장한다.

⑧ switch문을 벗어나 ⑨번으로 이동한다.

⑨ if (grade != 'F')

grade가 'F'가 아니면 ⑩번을 수행하고, 'F'이면 반복문의 처음인 ⑪번으로 이동한다. 현재 grade는 'B'이므로 ⑩번으로 이동한다.

- !=는 조건 판단문에 사용되는 예약어이므로 그대로 적는다.
- grade != 'F' : 조건식으로, 조건은 참(1) 또는 거짓(0)이 결과로 나올 수 있는 수식을 () 안에 입력한다.

⑩ printf("%d is %c %s\n", i + 1, grade, str);

i+1의 결과인 1을 정수(%d)로 출력한 후, 공백 한 칸과 is를 출력하고, 다시 공백 한 칸을 출력한다. 이어서 grade의 값 B를 문자(%c)로 출력하고 공백 한 칸을 출력한 후 str의 값 Rank를 문자열(%s)로 출력한다.

결과 1 is B Rank

이어서 제어문자 '\n'에 의해 커서가 다음 줄의 처음으로 이동된다.

for문의 반복이 아직 종료되지 않았으므로 제어는 ⑪번으로 이동한다.

⑪ 증가값(i++)에 의해 i는 1이 되고 최종값(i<5) 조건을 만족하므로 for의 { } 부분을 수행한다.

⑫ 현재 i가 1이므로 score[1]인 53을 10으로 나눈 결과 5에 해당하는 ⑬번으로 이동한다.

⑬ grade에 'F'를 저장한다.

⑭ grade가 'F'이므로 ⑩번을 수행하지 않고 ⑮번으로 이동한다.

위와 같은 과정을 for문의 i가 5가 되어 최종값(i<5) 조건을 만족하지 않을 때까지 3번 더 반복한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같습니다.

i	score[i]	score[i]/10	grade	grade != 'F'	출력
0	86	8	B	참	1 is B Rank
1	53	5	F	거짓	
2	95	9	A	참	3 is A Rank
3	76	7	C	참	4 is C Rank
4	61	6	F	거짓	
5					

결과 1 is B Rank
3 is A Rank
4 is C Rank

440318



25.11, 25.7, 25.4, 24.7, 23.4, 22.10, 22.5, 21.4, 20.11, 20.10, 필기 24.2, 23.7, 22.4, 22.3, 21.8

215 제어문 - Java



예제 다음 Java로 구현된 프로그램의 실행 결과를 확인하시오.

```
public class Test {
    public static void main(String[] args) {
        ① String str = "agile";
        ② int x[] = { 1, 2, 3, 4, 5 };
        ③ char y[] = new char[5];
        ④ int i = 0;
        ⑤ while (i < str.length()) {
        ⑥     y[i] = str.charAt(i);
        ⑦     i++;
        }
        ⑧ for (int p : x) {
        ⑨     i--;
        ⑩     System.out.print(y[i]);
        ⑪     System.out.print(p + " ");
        }
    }
}
```

코드 해설

❶ 문자열 변수 `str`를 선언하고 "agile"로 초기화한다.

❷ `int x[] = { 1, 2, 3, 4, 5 };`

5개의 요소를 갖는 정수 배열 `x`를 선언하고 초기화한다. 개수를 지정하지 않으면, 초기값으로 지정된 수만큼 배열의 요소가 만들어진다.

	[0]	[1]	[2]	[3]	[4]
x	1	2	3	4	5

❸ `char y[] = new char[5];`

5개의 요소를 갖는 문자 배열 `y`를 선언한다.

	[0]	[1]	[2]	[3]	[4]
y					

- Java에서 배열은 객체로 취급되며, 객체 변수는 'new' 명령을 이용해서 생성해야 한다. 배열을 선언하면서 초기값을 지정하는 경우에는 ❷번과 같이 선언하면서 지정하면 된다.
- `char y[]`: `y`는 문자형 배열 변수라는 의미이다. Java에서는 '`char[] y`'와 같이 대괄호를 자료형 바로 뒤에 적는 것을 선호하는데, C 언어에서는 이렇게 표기할 수 없다.
- `new char[5]`: 5개의 요소를 갖는 문자형 배열을 생성한다.

❹ 정수형 변수 `i`를 선언하고 0으로 초기화한다.

❺ `while (i < str.length()) {`

`i`가 `str` 변수의 길이보다 작은 동안 ❹~❺번을 반복 수행한다.

- `while`은 반복문에서 사용되는 예약어로 그대로 입력한다.
- `(i < str.length())`: 조건으로, 참이나 거짓을 결과로 갖는 수식을 조건에 입력한다. 참(1)을 직접 입력할 수도 있다.
- `length()` 메소드: 변수의 크기를 반환한다. `str`에는 5글자의 문자열이 저장되어 있으므로 `str.length()`는 5이다.

❻ `y[i] = str.charAt(i);`

`y[i]`에 `str`의 `i`번째에 있는 문자를 저장한다.

❼ `'i = i + 1';`과 동일하다. `i`의 값을 1씩 누적시킨다.

❹~❺번 반복문의 수행에 따른 변수들의 변화는 다음과 같다.

i	str	str.length()	str.charAt(i)	y[]
0	agile	5	a	a
1			g	a g
2			i	a g i
3			l	a g i l
4			e	a g i l e
5				

❻ `for (int p : x) {`

향상된 반복문이다. `p`는 `x` 배열의 각 요소의 값을 차례로 받으면서 `x` 배열의 요소 수만큼 ❹~❺번을 반복 수행한다. `x` 배열이 5개의 요소를 가지므로 각 요소를 `p`에 저장하면서 ❹~❺번을 5회 수행한다.

- `int p : x` 배열의 각 요소를 일시적으로 저장하기 위해 선언한 변수이다. `x` 배열과 형이 같아야 한다. `x` 배열이 정수면 정수, 문자면 문자여야 한다.
- `x`: 배열의 이름이다.

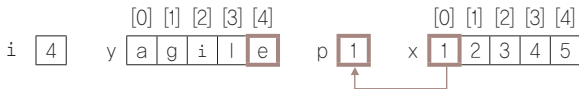
❹ `'i = i - 1';`과 동일하다. `i`의 값을 1씩 감소시킨다.

❺ `y[i]`의 값을 출력한다.

❻ `p`의 값을 출력하고, 이어서 공백 한 칸을 출력한다.

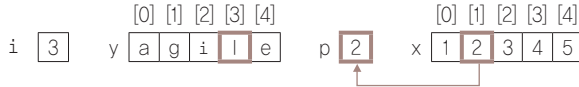
변수의 변화는 다음과 같다. ❹번 반복문을 수행한 후 `i`는 5였다가 ❹번을 수행한 후 `i`는 4가 되었다.

- ❹~❺번의 첫 번째 수행: `x` 배열의 첫 번째 값이 `p`를 거쳐 `y[4]`의 값과 함께 출력된다.



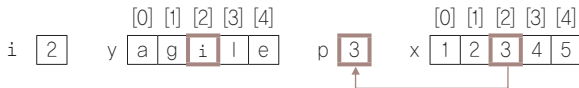
결과 e1

- ⑧~⑪의 두 번째 수행 : x 배열의 두 번째 값이 p를 거쳐 y[3]의 값과 함께 출력된다.



결과 e1 l2

- ⑧~⑪의 세 번째 수행 : x 배열의 세 번째 값이 p를 거쳐 y[2]의 값과 함께 출력된다.



결과 e1 l2 i3

이런 방식으로 x 배열의 요소 수만큼 2번 더 반복한다.

- ⑧~⑪번 반복문의 수행에 따른 변수들의 변화는 다음과 같다.

p	i	y[i]	출력
1	4	e	e1
2	3	l	e1 l2
3	2	i	e1 l2 i3
4	1	g	e1 l2 i3 g4
5	0	a	e1 l2 i3 g4 a5

440319



216 break와 continue

25.4, 23.4, 20.10, 필기 24.2, 23.2, 21.3



- `break` : 반복문이나 `switch`문 안에서 `break`가 나오면 블록을 벗어남
- `continue` : 반복문에서 `continue`가 나오면 `continue` 이후의 문장을 실행하지 않고 제어를 반복문의 처음으로 옮김



예제 다음 C언어로 구현된 프로그램의 실행 결과를 확인하시오.

```
#include <stdio.h>

main() {
    ① int a = 50;
    ② int *b = &a;
    ③ *b = *b + 20;
    ④ printf("%d, %d\n", a, *b);
    ⑤ char *s;
    ⑥ s = "gilbut";
    ⑦ for (int i = 0; i < 6; i += 2) {
    ⑧     printf("%c, ", s[i]);
    ⑨     printf("%c, ", *(s + i));
    ⑩     printf("%s\n", s + i);
    }
}
```

코드 해설

- ① 정수형 변수 a를 선언하고 50으로 초기화한다.

※ 주기억장치의 빈 공간 어딘가에 a라는 이름을 붙이고 그곳에 50을 저장합니다.(여기서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 주소를 실제 표현되는 16진수가 아니라 10진수로 표현했습니다.)

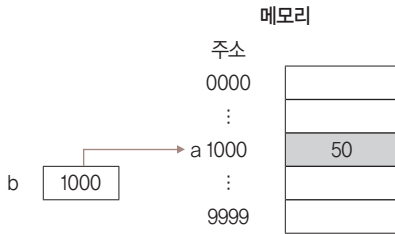
메모리	
주소	
0000	
⋮	
a 1000	50
⋮	
9999	

- ② int *b = &a;

정수형 변수가 저장된 곳의 주소를 기억할 포인터 변수 b를 선언하고, a의 주소로 초기화한다. b에는 a의 주소가 저장된다.

- int : 포인터 변수가 가리키는 곳에 저장되는 값의 자료형을 입력한다. 정수형 변수 a가 저장된 곳의 주소를 기억할 것이므로 int를 사용한다.
- *b : 포인터 변수를 선언할 때는 변수명 앞에 *를 붙인다.
- &a : 변수의 주소를 알아낼 때는 변수 앞에 번지 연산자 &를 붙인다.

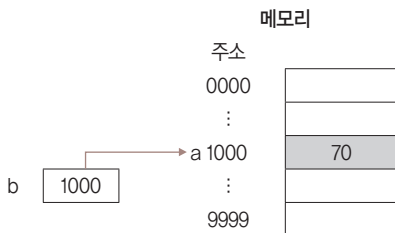
※ 변수 a의 주소가 b에 기억된다는 것은 b가 변수 a의 주소를 가리키고 있다는 의미입니다.



③ *b = *b + 20;

b가 가리키는 곳의 값(*b)에 20을 더한다. b가 가리키는 곳이 a이므로 결국 a의 값도 바뀌는 것이다. 실행문에서 포인터 변수에 간접 연산자 *를 붙이면 해당 포인터 변수가 가리키는 곳의 값을 말한다.

※ b가 가리키는 곳의 값에 20을 더해 다시 b가 가리키는 곳에 저장합니다. 그곳은 변수 a의 주소이므로 변수 a의 값도 저절로 변경되는 것입니다.



④ printf("%d, %d\n", a, *b);

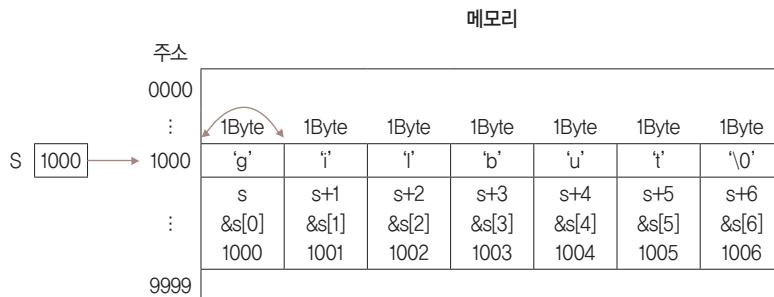
a의 값을 정수로 출력한 후 쉼표(,)와 공백 한 칸을 출력한다. 이어서 b가 가리키는 곳의 값(*b)을 정수로 출력한다.

결과 70, 70

⑤ 문자형 변수가 저장된 곳의 주소를 기억할 포인터 변수 s를 선언한다.

⑥ s = "gilbut";

• s는 주소를 기억하는 포인터 변수이므로 s에 "gilbut"가 기억되는 것이 아니라 "gilbut"라는 문자열이 메모리의 어딘가에 저장된 후 그 저장된 곳의 주소가 s에 기억된다.



• 문자열을 저장하는 경우 문자열의 끝을 의미하는 널 문자('\0')가 추가로 저장되며, 출력 시 널 문자는 표시되지 않는다.

⑦ 반복 변수 i가 0부터 2씩 증가하면서 6보다 작은 동안 ⑧~⑩번을 반복 수행한다. i가 0인 상태에서 ⑧번으로 이동한다.

⑧ printf("%c, ", s[i]);

s[i], 즉 s[0]의 값을 문자로 출력한 후, 쉼표(,)와 공백 한 칸을 출력한다.

결과 g,

⑨ printf("%c, ", *(s + i));

(s+i), 즉 (s+0)이 가리키는 곳의 값을 문자로 출력한 후, 쉼표(,)와 공백 한 칸을 출력한다.

결과 g, g,

⑩ printf("%s\n", s + i);

(s+i), 즉 (s+0)의 위치부터 문자열의 끝('\0')까지의 모든 문자를 하나의 문자열로 출력하고 커서를 다음 줄의 처음으로 옮긴다.

결과 **g, g, gilbut**

※ ⑦~⑩번 반복문의 수행에 따른 변수들의 변화는 다음과 같습니다.

i	s[i]	s+i	*(s+i)	출력
0	g	1000	g	g, g, gilbut
2	l	1002	l	g, g, gilbut l, l, lbut
4	u	1004	u	g, g, gilbut l, l, lbut u, u, ut
6				

440321



218 구조체 - C언어

25.11, 25.7, 25.4, 24.10, 24.7, 23.10, 22.7, 21.10, 21.4, 필기 24.5, 23.5, 23.2, 22.4



예제 다음 C언어로 구현된 프로그램의 실행 결과를 확인하시오.

```
#include <stdio.h>
```

A struct jsu { 구조체 jsu를 정의한다.

char nae[12]; 12개의 요소를 갖는 문자 배열 nae를 선언한다.

int os, db, hab, hhab; 정수형 변수 os, db, hab, hhab를 선언한다.

```
};
```

```
int main() {
```

① struct jsu st[3] = { {"데이터1", 95, 88}, {"데이터2", 84, 91}, {"데이터3", 86, 75} };

② struct jsu* p;

③ p = &st[0];

④ (p + 1)->hab = (p + 1)->os + (p + 2)->db;

⑤ (p + 1)->hhab = (p + 1)->hab + p->os + p->db;

⑥ printf("%d", (p + 1)->hab + (p + 1)->hhab);

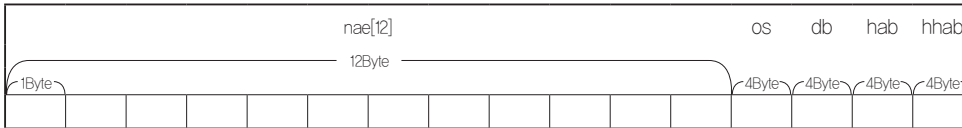
```
}
```

코드 해설

A 구조체 jsu의 구조



※ 위의 구조체는 다음과 같이 메모리의 연속된 공간에 저장된 후 사용됩니다.



모든 C 프로그램은 반드시 main() 함수에서 시작한다.

1 구조체 jsu 자료형으로 3개짜리 배열 st를 선언하고 초기화한다.

	char nae[12]	int os	int db	int hab	int hhab
st[0]	st[0].nae[0]~st[0].nae[11]	st[0].os	st[0].db	st[0].hab	st[0].hhab
st[1]	st[1].nae[0]~st[1].nae[11]	st[1].os	st[1].db	st[1].hab	st[1].hhab
st[2]	st[2].nae[0]~st[2].nae[11]	st[2].os	st[2].db	st[2].hab	st[2].hhab

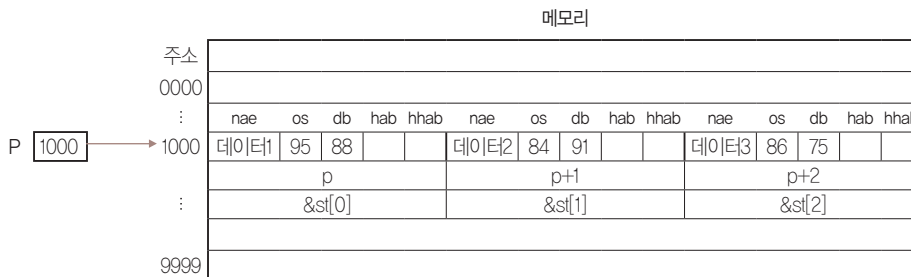


	char nae[12]	int os	int db	int hab	int hhab
st[0]	데 이 터 1 \0	95	88		
st[1]	데 이 터 2 \0	84	91		
st[2]	데 이 터 3 \0	86	75		

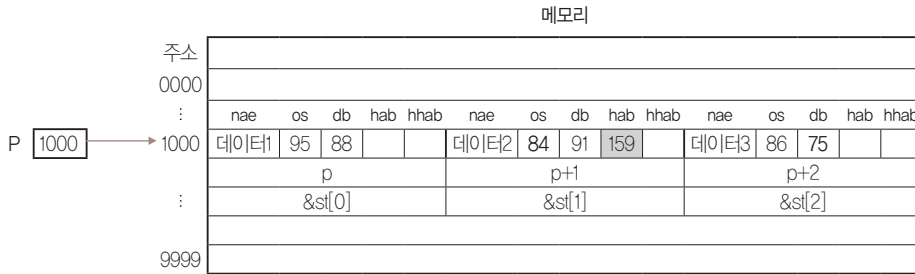
※ 문자열을 저장하는 경우 문자열의 끝을 의미하는 널 문자(\0)가 추가로 저장되며, 출력 시 널 문자는 표시되지 않습니다. 영문, 숫자는 1Byte, 한글은 2Byte를 차지합니다.

2 구조체 jsu의 포인터 변수 p를 선언한다.

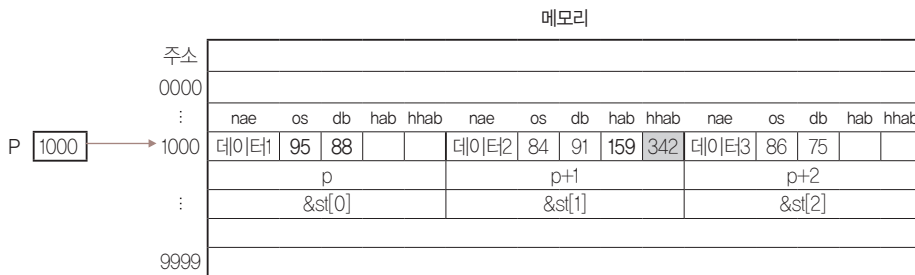
3 p에 st 배열의 첫 번째 요소의 주소를 저장한다.



4 p+1이 가리키는 곳의 멤버 hab에 p+1이 가리키는 곳의 멤버 os 값과 p+2가 가리키는 곳의 멤버 db 값을 더한 후 저장한다. p가 st[0]을 가리키므로 p+1은 st[1]을 p+2는 st[2]를 가리킨다. 따라서 st[1]의 os 값 84와 st[2]의 db 값 75를 더한 값 159를 st[1]의 hab에 저장한다.



- ⑤ p+1이 가리키는 곳의 멤버 hhab에 p+1이 가리키는 곳의 멤버 hab 값과 p가 가리키는 곳의 멤버 os와 db 값을 모두 더한 후 저장한다. st[1]의 hab 값 159, st[0]의 os와 db 값 95와 88을 모두 더한 값 **342**를 st[1]의 hhab에 저장한다.



- ⑥ p+1이 가리키는 곳의 멤버 hab와 hhab의 값을 더한 후 정수로 출력한다. 159와 342를 더한 **501**이 출력된다.

결과 501

440322



219 사용자 정의 함수 - C언어

24.10, 24.7, 24.4, 23.10, 23.7, 23.4, 22.10, 22.7, 22.5, 21.7, 20.10, 20.5, 필기 22.4, 22.3



예제 다음은 재귀 함수를 이용해 팩토리얼(Factorial) 연산을 수행하는 C 프로그램이다. 그 실행 결과를 확인하시오.

```
#include <stdio.h>

int factorial(int n);

main() {
    int (*pf)(int);
    pf = factorial;
    printf("%d", pf(3));
}
```

사용할 사용자 정의 함수를 선언하는 곳이다. main() 함수에서 사용할 factorial이란 함수를 이곳에서 정의하는 것이다. 즉 factorial 함수를 프로그램에서 만들어 사용하겠다는 의미이다.

- int : 사용할 함수의 반환값이 정수임을 알려준다. 그대로 적어준다.
- factorial : 함수의 이름이다. main() 함수 뒤에서 정의한 이름과 일치해야 한다.
- (int n) : 인수를 저장할 변수이다. 호출하는 곳에서 보내준 인수와 자료형이 일치해야 한다.

```
int factorial(int n) {
    if (n <= 1)
        return 1;
    else
        return n * factorial(n - 1);
}
```

코드 해설

모든 C 프로그램은 반드시 main() 함수에서 시작한다.

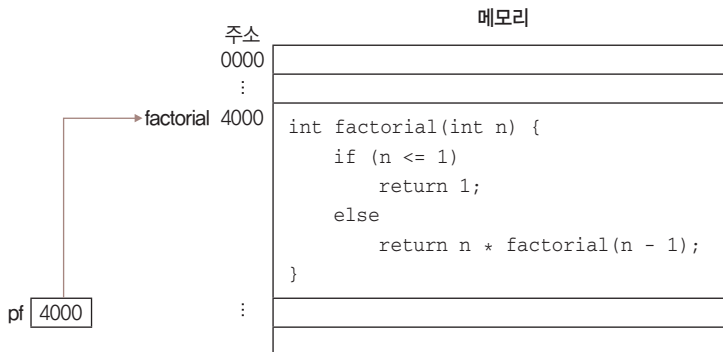
```
main() {
  ❶ int (*pf)(int);
  ❷ pf = factorial;
  ❸ printf("%d", pf(3));
}
```

❶ int (*pf)(int);

정수형 인수 한 개를 받는 정수형 함수 포인터 pf를 선언한다.

- int : 함수의 반환값이 정수임을 알려준다.
- (*pf) : 함수가 저장된 곳의 위치를 저장하는 함수 포인터의 이름이다.
- (int) : 함수가 전달받을 인수의 자료형이다.

❷ factorial 함수의 시작 주소를 함수 포인터 pf에 저장한다. 즉 pf가 factorial() 함수의 시작 주소를 가리키고 있다는 것을 의미한다. 다음 그림에서 factorial() 함수가 할당된 공간의 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.



❸ 3을 인수로 하여 pf() 함수를 호출한 다음 돌려받은 값을 정수형으로 출력한다. pf에는 factorial() 함수의 시작 주소가 저장되어 있으므로 함수 포인터 pf를 호출하는 것은 factorial() 함수를 호출하는 것과 같은 의미이다.

```
  ❹ int factorial(int n) {
  ❺     if (n <= 1)
        return 1;
    else
  ❻         return n * factorial(n - 1);
  }
```

④ int factorial(int n) {

정수를 반환하는 factorial 함수의 시작점이다. ③번에서 전달받은 3을 정수형 변수 n이 받는다.

- int : 함수의 반환값이 정수임을 알려준다.
- factorial : 함수의 이름이다. main() 함수 앞에서 선언한 이름과 일치해야 한다.
- (int n) : 호출하는 곳에서 보내준 인수를 저장할 변수이다. 호출하는 곳에서 보내준 인수와 자료형이 일치해야 한다.

⑤ factorial 함수가 호출될 때 3을 전달받았으므로 n은 3이다. n의 값 3은 1보다 작거나 같지 않으므로 ⑥번을 수행한다.

⑥ return n * factorial(n - 1);

n * factorial(n-1)을 연산한 후 함수를 호출했던 ③번으로 결과를 반환한다. ⑥번을 수행하기 위해 factorial() 함수를 호출하는데, 호출할 때 전달되는 값은 factorial(n-1)이므로 factorial(2)인 상태로 호출된다.

```
⑦ int factorial(int n) {  
⑧     if (n <= 1)  
        return 1;  
    else  
⑨        return n * factorial(n - 1);  
}
```

factorial 함수가 호출될 때 2를 전달받았으므로 n은 2이다. ⑧번의 조건을 만족하지 않으므로 ⑨번을 수행한다. ⑨번을 수행하기 위해 factorial() 함수를 호출하는데, 호출할 때 전달되는 값은 factorial(n-1)이므로 factorial(1)인 상태로 호출된다.

```
⑩ int factorial(int n) {  
⑪     if (n <= 1)  
⑫         return 1;  
    else  
        return n * factorial(n - 1);  
}
```

factorial 함수가 호출될 때 1을 전달받았으므로 n은 1이다. ⑪번의 조건을 만족하므로 ⑫번을 수행한다. 'return 1;'이므로 함수의 실행을 종료하고 1을 반환하면서 제어를 factorial(1) 함수를 호출했던 ⑬번으로 옮긴다.

```
⑦ int factorial(int n) {  
⑧     if (n <= 1)  
        return 1;  
    else  
⑨⑬    return n * factorial(n - 1);  
}
```

⑬ return n * factorial(n - 1);

⑫번에서 1을 반환하였으므로 2를 반환하면서 제어를 factorial(2) 함수를 호출했던 ⑭번으로 옮긴다.

return n * factorial(n - 1);

② ⑥

- ② : 2 ('factorial(n-1)'을 호출할 때 n은 2였으므로)
- ⑥ : 1 (⑫번에서 1을 반환하였으므로)

```
④ int factorial(int n) {  
⑤     if (n <= 1)  
        return 1;  
    else  
⑥⑭    return n * factorial(n - 1);  
}
```

⑭ return n * factorial(n - 1);

⑬번에서 2를 반환하였으므로 6을 반환하면서 제어를 factorial(3) 함수를 호출했던 ⑮번으로 옮긴다.

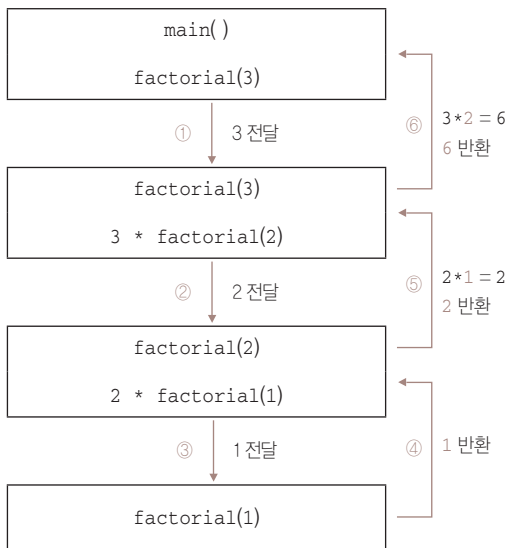
return $\frac{n}{3} * \frac{\text{factorial}(n-1)}{2}$;

```
main() {  
  ① int (*pf)(int);  
  ② pf = factorial;  
  ③⑤ printf("%d", pf(3));  
}
```

⑮ ⑭번에서 6을 반환하였으므로 돌려받은 값 6을 정수형으로 출력하고 프로그램을 종료한다.

결과 6

지금까지의 재귀 함수 과정을 개괄적인 그림을 통해 살펴보자.





예제 다음은 Java에서 클래스를 만들고 객체를 생성해서 사용하는 간단한 프로그램이다. 어떤 일을 수행하는지 확인하시오.

```

A class ClassA {
B     int a = 10;
C     ④ int funcAdd(int x, int y) {
        ⑤     return x + y + a;
        }
    }

D public class Test {
E     public static void main(String[] args) {
        ①     int x = 3, y = 6, r;
        ②     ClassA cal = new ClassA();
        ③⑥    r = cal.funcAdd(x, y);
        ⑦     System.out.print(r);
    }
}

```

코드 해설

- A** class ClassA {
ClassA 클래스를 정의한다. **B**~**C**가 클래스의 범위이다.
- class : 클래스를 정의하는 명령어로, 꼭 써야 하는 예약어이다. 같은 파일에서 클래스를 정의할 때는 public을 두 번 사용하지 못한다. 실행 클래스에서 사용하므로 여기서는 사용할 수 없다. 그렇다는 것이다. 외우지는 마라.
 - ClassA : 클래스 이름으로, 사용자가 원하는 이름을 임의로 지정할 수 있다. 단 첫 글자는 관례상 대문자로 지정한다.
- B** 정수형 변수 a를 선언하고 10으로 초기화한다. Java에서는 클래스 안에 선언하는 변수를 클래스의 속성이라고 부른다.
- C** int funcAdd(int x, int y) {
정수를 반환하는 funcAdd(int x, int y) 메소드를 정의한다. 호출문으로부터 정수형 인수 2개를 전달받아 각각 x와 y에 저장한다.
- int : 메소드의 반환값이 정수임을 알려준다.
 - funcAdd : 메소드의 이름이다.
 - (int x, int y) : 호출하는 곳에서 보내준 인수를 저장할 변수이다. 호출하는 곳에서 보내준 인수의 개수와 자료형이 일치해야 한다.
- D** Test 클래스를 정의한다. 실행 클래스의 시작점으로 Java 프로그램은 아무리 작은 프로그램이라도 클래스를 만들어서 클래스 안에 실행문과 메소드(함수)를 만들고 실행해야 한다. 그리고 클래스 중에는 반드시 main() 메소드를 담고 있는 실행 클래스가 있어야 한다.
- E** main() 메소드의 시작점이다. 여기서부터 실제 프로그램이 시작된다.
- ① 정수형 변수 x, y, r을 선언하고, x와 y를 각각 3과 6으로 초기화한다.
- ② ClassA cal = new ClassA();

ClassA 클래스의 객체 변수 cal을 선언한다.

- ClassA : 클래스의 이름이다. 앞에서 정의한 클래스의 이름을 그대로 적어준다.
- cal : 객체 변수의 이름이다. 사용자가 원하는 이름을 적으면 된다.
- new : 객체 생성 예약어이다. 그대로 적어준다.
- ClassA() : 생성자이다.

③ x와 y의 값 3과 6을 인수로 cal의 funcAdd() 메소드를 호출하여 반환받은 값을 r에 저장한다.

cal은 ClassA 클래스의 객체 변수이므로 ClassA의 funcAdd() 메소드인 ④번이 호출된다.

④ 정수를 반환하는 funcAdd() 메소드의 시작점이다. 호출문으로부터 정수형 인수 2개를 전달받아 각각 x와 y에 저장한다. ③번에서 호출할 때 3과 6을 전달했으므로 x는 3, y는 6이다.

⑤ $x + y + a$ 를 연산한 후 메소드를 호출했던 ⑥번으로 결과를 반환한다. $x + y$ 의 값은 9이고, a는 메소드에 없으므로 소속된 클래스에서 찾는다. ClassA의 a의 값이 100이므로 $x + y + a(3 + 6 + 10)$ 의 값은 19가 된다.

⑥ ⑤번에서 19가 반환되었으므로 r에 19를 저장한다.

⑦ r의 값 19를 출력한다.

결과 19

340395



221 Java의 클래스 2

25.11, 25.7, 25.4, 24.10, 24.7, 23.4, 22.7, 21.7, 20.10, 20.7, 20.5



예제 다음 Java 프로그램의 실행 결과를 확인하시오.

```
A class ClassA {  
B ⑤ ClassA() {  
    ⑥ System.out.print('A');  
    ⑦ this.prn();  
    ⑩ }  
C void prn() {  
    System.out.print('B');  
}  
}  
D class ClassB extends ClassA {  
E ③ ClassB() {  
    ④ super();  
    ⑪ System.out.print('D');  
}  
F ⑧ void prn() {  
    ⑨ System.out.print('E');  
}
```

```

G 13 void prn(int x) {
    14     System.out.print(x);
    }
}

public class Test {
    public static void main(String[] args) {
        1 int x = 7;
        2 ClassB cal = new ClassB();
        12 cal.prn(x);
        15 }
    }
}

```

코드 해설

A class ClassA {

ClassA 클래스를 정의한다. B~C가 클래스의 범위이다.

B ClassA() {

ClassA 클래스에 속한 ClassA() 메소드를 정의한다. ClassA() 메소드는 클래스와 이름이 동일하다. 이와 같이 클래스와 이름이 동일한 메소드는 해당 클래스의 객체 변수 생성 시 자동으로 실행되는데, 이러한 메소드를 생성자(Constructor)라고 한다.

C void prn() {

반환값 없는 메소드 prn()을 정의한다.

D class ClassB extends ClassA {

ClassB를 클래스 정의하고 부모 클래스로 ClassA를 지정하면서 ClassA에 속한 변수와 메소드를 상속받는다. ClassB 클래스는 ClassA의 변수와 메소드를 사용할 수 있게 된다. E~G가 클래스의 범위이다.

• extends [클래스명] : 클래스 정의 시 상속받을 클래스를 추가하는 예약어

E ClassB() {

ClassB 클래스에 속한 ClassB() 메소드를 정의한다. ClassB() 메소드도 ClassB 클래스와 이름이 동일하므로 객체 변수 생성 시 자동으로 실행되는 생성자이다.

F void prn() {

반환값 없는 메소드 prn()을 정의한다. D에서 ClassB 클래스는 ClassA 클래스의 메소드를 사용할 수 있다고 했으므로 ClassB 클래스에는 이름이 같은 메소드(C, F) prn()이 두 개 존재하게 된다. 이와 같이 상속으로 인해 동일한 이름의 메소드가 여러 개인 경우, 부모 클래스에서 정의된 prn() 메소드(C)는 자식 클래스의 prn() 메소드(F)에 의해 재정의되어 자식 클래스의 prn 메소드(F)만 사용되는데, 이를 메소드 오버라이딩 또는 메소드 재정의라고 한다.

G void prn(int x) {

반환값 없는 메소드 prn(int x)를 정의한다. 메소드의 이름이 C, F와 같지만 '인수를 받는 자료형과 개수'가 다르므로 서로 다른 메소드이다. 즉 prn()과 prn(int x)는 다른 메소드라는 것이다. 이렇게 이름은 같지만 인수를 받는 자료형과 개수를 달리하여 여러 기능을 정의하는 것을 오버로딩(Overloading)이라고 한다.

모든 Java 프로그램의 실행은 반드시 main() 메소드에서 시작한다.

1 정수형 변수 x를 선언하고 7로 초기화한다.

2 ClassB cal = new ClassB();

ClassB 클래스의 객체 변수 cal을 선언하고 ClassB 클래스의 생성자를 호출한다. ClassB 클래스에는 클래스명과 동일한 생성자가 정의되어 있으므로 생성자를 실행하기 위해 ❸번으로 이동한다.

- ClassB : 클래스의 이름이다. 앞에서 정의한 클래스의 이름을 그대로 적어준다.
- cal : 객체 변수의 이름이다. 사용자가 원하는 이름을 적으면 된다.
- new : 객체 생성 예약어다. 그대로 적어준다.
- ClassB() : 클래스와 이름이 동일한 메소드로, 생성자이다.

❸ ClassB 클래스의 생성자인 ClassB() 메소드의 시작점이다. 지금까지 클래스 안에 생성자를 정의하는 문장이 있을 경우 객체가 생성될 때 자동으로 호출되어 실행된다.

❹ 부모 클래스의 생성자인 ClassA() 메소드를 호출한다. ❻번으로 이동한다.

- super() : 부모 클래스의 생성자를 호출한다.

❺ ClassA 클래스의 생성자 ClassA() 메소드의 시작점이다.

❻ A를 출력한다.

결과 A

❼ 자신이 속한 ClassA 클래스의 prn() 메소드를 호출한다. ClassA 클래스의 prn() 메소드는 ClassB 클래스의 prn() 메소드에 의해 재정의되었으므로 ❸번으로 이동한다.

❸ 반환값 없는 prn() 메소드의 시작점이다.

❾ E를 출력한 후 메소드가 종료되면 호출했던 ❷번의 다음 줄인 ❿번으로 이동한다.

결과 AE

❿ ClassA() 메소드가 종료되었으므로 호출했던 ❹번의 다음 줄인 ⓫번으로 이동한다.

⓫ D를 출력한 후 ClassB() 메소드가 종료되면 호출했던 ❷번의 다음 줄인 ⓬번으로 이동한다.

결과 AED

⓬ x의 값 7을 인수로 cal의 prn(int x) 메소드를 호출한다. ⓭번으로 이동한다.

⓭ 반환값 없는 prn(int x) 메소드의 시작점이다. ⓬번에서 전달한 7을 x가 받는다.

⓬ x의 값 7을 출력한 후 prn(int x) 메소드가 종료되면 호출했던 ⓬번의 다음 줄인 ⓮번으로 이동하여 프로그램을 종료한다.

결과 AED7

340396



222 생성자

20,10



- 생성자(Constructor)는 객체 변수 생성에 사용되는 메소드로, 객체 변수를 생성하면서 초기화를 수행한다.
- 클래스 안에 생성자를 정의하는 문장이 있다면 문장에 적힌 대로 객체 변수를 초기화하면서 생성하지만, 없으면 그냥 객체 변수만 생성하고 제어가 다음 줄로 이동한다.



예제 다음 Java 프로그램의 실행 결과를 확인하시오.

```

A  abstract class Animal {
B      String a = " is animal";
C      abstract void look();
D      8 void show() {
        9      System.out.println("Zoo");
        }
    }
E  class Chicken extends Animal {
F      2 Chicken() {
        3      look();
        6  }
G      4 void look() {
        5      System.out.println("Chicken" + a);
        }
H      void display() {
        System.out.println("two wings");
    }
}

public class Test {
    public static void main(String[] args) {
        1  Animal a = new Chicken();
        7  a.show();
        10 }
    }

```

코드 해설

A abstract class Animal {

추상 클래스 Animal을 정의한다.

• abstract [클래스 정의부] : abstract는 추상 클래스를 정의하는 명령어로, 추상 클래스 정의 시 꼭 써야하는 예약어이다.

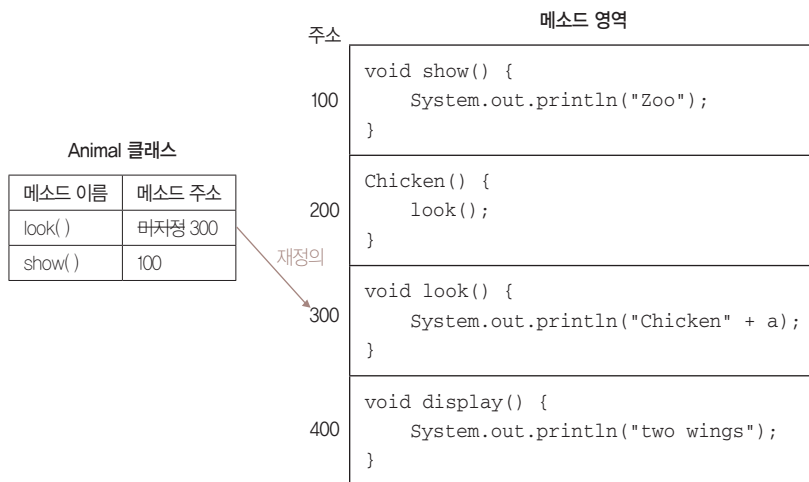
※ 추상 클래스는 내부에 실행 코드가 없는 추상 메소드를 포함하기 때문에 객체 변수의 생성자로 사용할 수 없습니다.

예 Animal a = new Animal(); ← 오류 발생

- B** 문자열 변수 `a`를 선언하고 " is animal"로 초기화한다.
- C** `abstract void look( );`
추상 메소드 `look( )`을 정의한다.
- **abstract [메소드 정의부]** : `abstract`는 추상 메소드를 정의하는 명령어로, 추상 메소드 정의 시 꼭 써야하는 예약어이다.
 - ※ 추상 메소드는 선언만 있고 내부에 실행 코드가 없는 메소드로, 이후 상속 관계가 설정된 자식 클래스에서 재정의한 후 사용합니다.
- D** 반환값 없는 메소드 `show( )`를 정의한다.
- E** 클래스 `Chicken`을 정의하고 부모 클래스로 `Animal`을 지정하면서 `Animal`에 속한 변수와 메소드를 상속받는다.
- F** `Chicken` 클래스에 속한 `Chicken( )` 메소드를 정의한다. `Chicken` 클래스와 이름이 동일하므로 객체 변수 생성 시 자동으로 실행되는 생성자이다.
- G** 반환값 없는 메소드 `look( )`을 정의한다. `look( )` 메소드는 **C**에서 정의된 추상 메소드(**C**)를 재정의하는 것이다.
- H** 반환값 없는 메소드 `display( )`를 정의한다.

모든 Java 프로그램은 반드시 `main( )` 메소드에서 시작한다.

- 1** `Animal a = new Chicken( );`
`Chicken` 클래스의 생성자를 이용하여 `Animal` 클래스의 객체 변수 `a`를 선언한다.
- **[부모클래스명] [객체변수명] = new [자식클래스생성자()]** : 부모 클래스의 객체 변수를 선언하면서 자식 클래스의 생성자를 사용하면 형 변환이 발생한다.
 - 이렇게 형 변환이 발생했을 때 부모 클래스와 자식 클래스에 동일한 속성이나 메소드가 있으면 자식 클래스의 속성이나 메소드로 재정의된다.
 - ※ 객체 변수 `a`는 `Animal` 클래스의 객체 변수입니다. `Animal` 클래스는 실행 코드가 없는 추상 메소드 `look( )`으로 인해 객체 변수의 생성이 불가능해야 하지만, 형 변환으로 인해 `look( )` 메소드가 `Chicken` 클래스에서 재정의되었으므로 객체 변수의 생성이 가능해진 것입니다.
- 2** `Chicken` 클래스의 생성자인 `Chicken( )` 메소드의 시작점이다. 클래스 안에 생성자를 정의하는 문장이 있으므로 `Chicken` 클래스의 객체가 생성될 때 자동으로 호출되어 실행된다.
- 3** `look( )` 메소드를 호출한다. **4**번으로 이동한다.
- `look( )` 메소드는 `a` 객체의 자료형이 `Animal`이므로 `Animal` 클래스의 `look( )`이라고 생각할 수 있지만 **1**번에서 클래스 형 변환이 발생하였고, `look( )` 메소드가 자식 클래스에서 재정의되었으므로 `Chicken` 클래스의 `look( )` 메소드가 수행된다.
- ※ `Animal` 클래스의 `look( )` 메소드는 실행 코드가 정의되지 않은 추상 메소드로, 코드를 실행할 때 실행 코드가 저장된 메모리 위치를 참조할 수 없는데, 재정의가 이루어지면 다음 그림과 같이 `Chicken` 클래스의 `look( )` 메소드를 가리키게 됩니다.



- 4** 반환값 없는 `look( )` 메소드의 시작점이다.
- 5** `Chicken`과 `a`의 값 `is animal`을 출력한 후 커서를 다음 줄의 처음으로 옮긴다. 이어서 메소드가 종료되면 호출했던 **3**번의 다음 줄인 **6**번으로 이동한다.

결과 Chicken is animal

- ※ ⑤에서 Chicken 클래스가 Animal 클래스의 변수와 메소드를 상속받았으므로 Animal 클래스의 a 변수의 값을 사용할 수 있습니다.
- ⑥ Chicken() 메소드가 종료되었으므로 호출했던 ①번의 다음 줄인 ⑦번으로 이동한다.
- ⑦ a의 show() 메소드를 호출한다. a는 Animal 클래스의 객체 변수이므로 Animal의 show() 메소드인 ⑧이 호출된다.
- ⑧ 반환값 없는 show() 메소드의 시작점이다.
- ⑨ Zoo를 출력한 후 메소드가 종료되면 호출했던 ⑦번의 다음 줄인 ⑩번으로 이동하여 프로그램을 종료한다.

결과 Chicken is animal
Zoo

340554



224 Python의 활용 1

25.7, 25.4, 24.10, 24.7, 24.4, 23.10, 23.7, 21.10, 21.4, 20.11, 필기 25.5, 25.2, 24.7, 24.5, 24.2, 21.8



예제 다음 Python으로 구현된 프로그램에 "xyz321-opq654"를 입력했을 때 그 실행 결과를 확인하시오.

```
① x, y = input('입력 :').split('-')
② a = [ 'abc123', 'def456', 'ghi789' ]
③ a.append(x)
④ a.append(y)
⑤ a.remove('def456')
⑥ print(a[1][-3:], a[2][:3], sep = ',')
⑦ for i in range(3, 6):
⑧     print(i, end = ' ')
```

코드 해설

- ① x, y = input("입력 :").split("-")
화면에 **입력 :** 이 출력되고 그 뒤에서 커서가 깜박거리며 입력을 기다린다. 키보드로 값을 입력하고 엔터를 누르면 입력된 값이 split로 지정된 분리문자인 '-'를 기준으로 xyz321은 변수 x에 저장되고, opq654는 변수 y에 저장된다.
 - x, y : 입력되는 값이 저장될 변수이다.
 - input().split("-") : 분리문자 '-'를 기준으로 입력된 값을 변수 x와 y에 나눠서 저장한다.
 - '입력 :': 입력 시 화면에 출력되는 문자로, 생략이 가능하다.
- ② a = ['abc123', 'def456', 'ghi789']
리스트 a를 선언하면서 초기값을 지정한다. 초기값으로 지정된 수만큼 리스트의 요소가 만들어진다.

[0]	[1]	[2]
'abc123'	'def456'	'ghi789'
- ③ a.append(x)
a 리스트의 마지막에 x의 값 "xyz321"을 추가한다.

[0]	[1]	[2]	[3]
'abc123'	'def456'	'ghi789'	'xyz321'

④ a.append(y)

a 리스트의 마지막에 y의 값 'opq654'를 추가한다.

[0]	[1]	[2]	[3]	[4]
'abc123'	'def456'	'ghi789'	'xyz321'	'opq654'

⑤ a.remove('def456')

a 리스트에서 "def456"을 찾아 삭제하고, 이후의 요소들을 하나씩 앞으로 이동시킨다.

[0]	[1]	[2]	[3]
'abc123'	'ghi789'	'xyz321'	'opq654'

⑥ print(a[1][-3:], a[2][-3:], sep = ',')

a[1]과 a[2]의 요소들을 슬라이스(slice)하고, 쉼표(,)로 구분하여 출력한다. 이어서 커서를 다음 줄의 처음으로 옮긴다.

- a[1][-3:] : a[1]에 저장된 문자열 "ghi789"의 -3 위치부터 마지막 요소까지 추출한다.

a[1]	g	h	i	7	8	9
	-6	-5	-4	-3	-2	-1

※ 최종위치가 생략된 경우 초기위치로 지정된 위치부터 마지막 요소까지의 모든 값이 반환됩니다.

- a[2][-3:] : a[2]에 저장된 문자열 "xyz321"의 맨 처음 요소부터 -4까지의 요소들을 추출한다.

	0	1	2	3	4	5
a[2]	x	y	z	3	2	1
	-6	-5	-4	-3	-2	-1

※ 초기위치가 생략된 경우 요소의 맨 처음 요소부터 최종위치(-3)에서 1을 뺀 위치(-4)까지의 모든 값이 반환됩니다.

- sep = ',': 분리문자로 쉼표(,)를 지정한다. 출력할 값들을 쉼표로 구분하여 출력한다.

결과 789,xyz

⑦ for i in range(3, 6):

3에서 5(6-1)까지 순서대로 i에 저장하며 ③번을 반복 수행한다.

- i : 반복 변수로 range에서 생성되는 값을 순서대로 저장한다.
- in : for와 함께 사용되는 예약어로 그대로 입력한다.
- range(3, 6) : 3부터 5(6-1)까지의 연속적인 숫자를 생성한다.
- 콜론(:) : for문이 반복할 코드가 다음 줄부터 시작한다는 의미이다. 반드시 입력해야 한다.

⑧ print(i, end = '')

i의 값을 출력하고 종료문자로 공백 한 칸이 출력된다. ⑦번에서 i는 3에서 5까지 순서대로 저장한다고 하였으므로, 3 4 5가 출력된다.

789,xyz

결과 3 4 5

440328



225 Python의 활용 2

25.11, 25.7, 25.4, 23.4, 22.7, 20.7, 필기 22.3



예제 다음 Python 프로그램의 실행 결과를 확인하시오.

```
① a = {'apple', 'lemon', 'banana'}
② a.update( {'kiwi', 'banana'} )
③ a.remove('lemon')
④ a.add('apple')
⑤ for i in a:
⑥     print("과일명 : %s" % i)
```

코드 해설

세트는 수학에서 배우는 집합(set)과 동일한 역할을 하는 Python의 자료형으로, 중괄호{ }를 이용하여 리스트와 같이 다양한 요소들을 저장할 수 있다. 세트는 순서가 정해져 있지 않으며(unordered), 중복된 요소는 저장되지 않는다는 특징이 있다.

❶ a = {'apple', 'lemon', 'banana'}

세트 a를 선언하면서 초기값을 지정한다. 초기값으로 지정된 수만큼 세트의 요소가 만들어진다.

a ['apple' | 'lemon' | 'banana']

❷ a.update({'kiwi', 'banana'})

a 세트에 새로운 세트를 추가하여 확장한다. 새로운 세트 {'kiwi', 'banana'}가 추가되어야 하지만 'banana'는 이미 a 세트에 있으므로 'kiwi'만 추가된다.

a ['apple' | 'lemon' | 'banana' | 'kiwi']

❸ a.remove('lemon')

a 세트에서 'lemon'을 제거한다.

a ['apple' | 'banana' | 'kiwi']

❹ a.add('apple')

a 세트에 'apple'을 추가한다. a 세트에는 이미 'apple'이 존재하므로 a 세트의 요소는 변하지 않는다.

❺ for i in a:

a 세트의 각 요소의 값을 차례로 i에 저장하면서 a 세트의 요소 수만큼 ❻번을 반복 수행한다.

- i : a 세트의 각 요소가 일시적으로 저장될 변수이다.
- a : 세트의 이름이다.

❻ 과일명 : 과 i의 값을 출력한 후 커서를 다음 줄의 처음으로 옮긴다. for문에 의한 변수의 변화는 다음과 같다.

- ❺~❻번의 첫 번째 수행 : a 세트의 첫 번째 값이 i를 거쳐 출력된다.

i ['apple'] a ['apple' | 'banana' | 'kiwi']

결과 과일명 : apple

- ❺~❻번의 두 번째 수행 : a 세트의 두 번째 값이 i를 거쳐 출력된다.

i ['banana'] a ['apple' | 'banana' | 'kiwi']

결과 과일명 : apple
 과일명 : banana

- ❺~❻번의 세 번째 수행 : a 세트의 세 번째 값이 i를 거쳐 출력된다.

i ['kiwi'] a ['apple' | 'banana' | 'kiwi']

결과 과일명 : apple
 과일명 : banana
 과일명 : kiwi



- 어떤 문제를 해결하기 위한 과정을 수학식으로 표현한 것을 람다 식(Lambda Expression)이라고 한다.
- 프로그래밍 언어에서 람다 식은 수학적 연산을 수행하는 함수나 메소드를 간소화할 때 사용한다.
- 형식

```
lambda 변수명 : 수학식
```

- 변수명 : 인수로 전달받은 값을 저장할 변수의 이름을 지정
- 수학식 : 수행할 연산을 하나의 식으로 풀어 입력
- 예를 들어 다음과 같은 코드가 있다고 가정한다.

```
def func(x):
    return x * x - 3
print(func(10))
```

결과 97

- 위의 func() 함수는 다음과 같이 람다 식으로 수정하여 사용할 수 있다.

```
func = lambda x : x * x - 3
print(func(10))
```

결과 97



예제 다음은 두 수를 교환하는 프로그램을 Python으로 구현한 것이다.

```
class Cls:    Cls 클래스 정의부의 시작점이다. 여기서부터 ⑦번까지가 클래스 정의부에 해당한다.
    x, y = 10, 20    Cls 클래스의 변수(속성) x와 y를 선언하고, 각각 10과 20으로 초기화한다.
    ④ def chg(self):
    ⑤         temp = self.x
    ⑥         self.x = self.y
    ⑦         self.y = temp
    ① a = Cls()
    ② print(a.x, a.y)
    ③ a.chg()
    ⑧ print(a.x, a.y)
```

코드 해설

- ❶ Cls 클래스의 객체 a를 생성한다. 객체 a는 Cls의 속성 x, y와 메소드 chg()를 갖는다.

- a : 사용자 정의 변수다. 사용자가 임의로 지정한다.
- Cls() : 클래스의 이름이다. 괄호()를 붙여 그대로 적는다.

	ax	ay
a	10	20

- ❷ a 객체의 속성 x와 y를 출력한다.
- 객체와 속성은 (마침표)로 연결한다.

결과 10 20

- ❸ a 객체의 메소드 chg를 호출한다. ❹번으로 이동한다.
- 객체와 메소드는 (마침표)로 연결한 후 괄호()를 붙여 적는다.
- ❹ a 객체의 메소드 chg의 시작점이다. 별도로 사용되는 인수가 없으므로 괄호()에는 self만 적는다.
- ❺ a 객체의 속성 x의 값을 temp에 저장한다.
- self : 메소드 안에서 사용되는 self는 자신이 속한 클래스를 의미한다.
 - self.x : ax와 동일하다.

	ax	ay
temp	10	
a	10	20

- ❻ a 객체의 속성 y의 값을 a 객체의 속성 x에 저장한다.

	ax	ay
temp	10	
a	20	20

- ❼ temp의 값을 a 객체의 속성 y에 저장한다. 메소드 chg가 종료되었으므로 메소드를 호출한 다음 문장인 ❸번으로 제어를 옮긴다.

	ax	ay
temp	10	
a	20	10

- ❽ a 객체의 속성 x와 y를 출력한다.

결과 10 20
20 10



228 Python의 클래스 없는 메소드 사용



예제 다음 프로그램의 실행 결과를 확인하시오.

```

③ def calc(x, y):
④     x *= 3
⑤     y /= 3
⑥     print(x, y)
⑦     return x

① a, b = 3, 12
② a = calc(a, b)
⑧ print(a, b)

```

코드 해설

- ① 변수 a와 b에 3과 12를 저장한다.
- ② a와 b의 값 즉 3과 12를 인수로 하여 calc 메소드를 호출한 결과를 a에 저장한다. ③번으로 이동한다.
- ③ 메소드 calc의 시작점이다. ②번에서 calc(a, b)라고 했으므로 x는 a의 값 3을 받고, y는 b의 값 12를 받는다.
- ④ $x = x * 3$ 이므로 x는 9가 된다.
- ⑤ $y = y / 3$ 이므로 y는 4가 된다.

⑥ 결과 9 4.0

- ⑦ x의 값을 반환한다. x의 값 9를 ②번의 a에 저장한 후 제어를 ⑧번으로 옮긴다.

⑧ 결과 9 4.0
9 12



229 Python - Range



- Range는 연속된 숫자를 생성하는 것으로, 리스트나 반복문에서 많이 사용된다.
- 형식

range(최종값)

0에서 '최종값'-1까지 연속된 숫자를 생성한다.

range(초기값, 최종값)

'초기값'에서 '최종값'-1까지 연속된 숫자를 생성한다.

range(초기값, 최종값, 증가값)

- '초기값'에서 '최종값'-1까지 '증가값'만큼 증가하면서 숫자를 생성한다.
- '증가값'이 음수인 경우 '초기값'에서 '최종값'+1까지 '증가값'만큼 감소하면서 숫자를 생성한다.



230 Python - 슬라이스



- 슬라이스(slice)는 문자열이나 리스트와 같은 순차형 객체에서 일부를 잘라(slicing) 반환하는 기능이다.
- 형식

객체명[초기위치:최종위치]

'초기위치'에서 '최종위치'-1까지의 요소들을 가져온다.

객체명[초기위치:최종위치:증가값]

- '초기위치'에서 '최종위치'-1까지 '증가값'만큼 증가하면서 해당 위치의 요소들을 가져온다.
- '증가값'이 음수인 경우 '초기위치'에서 '최종위치'+1까지 '증가값' 만큼 감소하면서 해당 위치의 요소들을 가져온다.

- 슬라이스는 일부 인수를 생략하여 사용할 수 있다.

객체명[:] 또는 객체명[::]

객체의 모든 요소를 반환한다.

객체명[초기위치:]

객체의 '초기위치'에서 마지막 위치까지의 요소들을 반환한다.

객체명[:최종위치]

객체의 0번째 위치에서 '최종위치'-1까지의 요소들을 반환한다.

객체명[:증가값]

객체의 0번째 위치에서 마지막 위치까지 '증가값'만큼 증가하면서 해당 위치의 요소들을 반환한다.



231 Python - 주요 메소드



형식	내용
22.5 pop(위치)	객체의 '위치'에 있는 값을 출력하고 해당 요소를 삭제함 예 [10, 11, 12].pop(1) → 11 출력 → [10, 12]
25.7 len()	객체 요소의 개수를 반환함 예 len({2, 7, 6}) → 3
22.5 extend(리스트)	리스트의 끝에 새로운 '리스트'를 추가하여 확장함 예 ['a', 'b'].extend(['c', 'd']) → ['a', 'b', 'c', 'd']
22.5 reverse()	리스트의 순서를 역순으로 뒤집음 예 [1, 2, 3].reverse() → [3, 2, 1]
25.7 set()	• 반복 가능한 객체를 세트 자료형으로 변환하거나 빈 세트를 생성함 • 중복된 값은 제거됨 예 a = [1, 2, 3, 4, 2, 1] → {1, 2, 3, 4} b = set(a)
25.7 add(값)	세트에 '값'을 추가함 예 a = {2, 7, 6} → {2, 7, 6, 9} a.add(9)
25.7 values()	딕셔너리의 모든 값을 반환함 예 a = {'name': 'kim', 'age': 30} → ['kim', 30] a.values()



필기 23.5, 23.2

stdio.h

- 데이터의 입 · 출력에 사용되는 기능들을 제공한다.
- 주요 함수 : printf, scanf, fprintf, fscanf, fclose, fopen 등

필기 25.2, 23.2

string.h

- 문자열 처리에 사용되는 기능들을 제공한다.
- 주요 함수 : strlen, strcpy, strcmp 등

필기 25.5, 21.5

stdlib.h

- 자료형 변환, 난수 발생, 메모리 할당에 사용되는 기능들을 제공한다.
- 주요 함수 : atoi, atof, srand, rand, malloc, free 등

25.7, 25.4, 필기 25.8, 24.5, 22.7

malloc() 함수

- 바이트 단위로 메모리 공간을 동적으로 할당한다.
- 메모리 할당이 불가능할 경우 NULL이 반환된다.

25.7, 25.4, 필기 24.5, 22.7

free() 함수

malloc() 함수에 의해 동적으로 할당된 메모리를 해제한다.



프로그램의 정상적인 실행을 방해하는 조건이나 상태를 예외(Exception)라고 하며, 이러한 예외가 발생했을 때 프로그래머가 해당 문제에 대비해 작성해 놓은 처리 루틴을 수행하도록 하는 것을 예외 처리(Exception Handling)라고 한다.

- JAVA는 잘못된 동작이나 결과에 영향을 줄 수 있는 예외를 객체로 취급하며, 예외와 관련된 클래스를 java.lang 패키지에서 제공한다.
- JAVA에서는 try ~ catch 문을 이용해 예외를 처리한다.
- try 블록 코드를 수행하다 예외가 발생하면 예외를 처리하는 catch 블록으로 이동하여 예외 처리 코드를 수행하므로 예외가 발생한 이후의 코드는 실행되지 않는다.
- 기본 형식

```
try {
    예외가 발생할 가능성이 있는 코드;
}
catch ( 예외객체1 매개변수 ) {
    예외객체1에 해당하는 예외 발생 시 처리 코드;
}
catch ( 예외객체2 매개변수 ) {
    예외객체2에 해당하는 예외 발생 시 처리 코드;
}
catch ( 예외객체n 매개변수 ) {
    예외객체n에 해당하는 예외 발생 시 처리 코드;
}
catch (Exception 매개변수) {
    예외객체1~n에 해당하지 않는 예외 발생 시 처리 코드;
}
finally {
    예외의 발생 여부와 관계없이 무조건 처리되는 코드;
}
```

※ 일반적으로 예외가 발생한 경우에는 'try문 → 해당 예외 catch문 → finally문' 순으로 진행되며, 예외가 발생하지 않은 경우에는 'try문 → finally문' 순으로 진행됩니다. finally문은 예외 발생과 관계없이 무조건 수행되는 블록으로 생략이 가능합니다.



234 JAVA의 주요 예외 객체



예외 객체	발생 원인
ClassNotFoundException	클래스를 찾지 못한 경우
NoSuchMethodException	메소드를 찾지 못한 경우
FileNotFoundException	파일을 찾지 못한 경우
InterruptedException	입 · 출력 처리가 중단된 경우
ArithmeticException	0으로 나누는 등의 산술 연산에 대한 예외가 발생한 경우
IllegalArgumentException	잘못된 인자를 전달한 경우
NumberFormatException	숫자 형식으로 변환할 수 없는 문자열을 숫자 형식으로 변환한 경우
ArrayIndexOutOfBoundsException	배열의 범위를 벗어난 접근을 시도한 경우
NegativeArraySizeException	0보다 작은 값으로 배열의 크기를 지정한 경우
NullPointerException	존재하지 않는 객체를 참조한 경우